

Organización de Datos

Resumen explicativo — Organización de Archivos Directos y Hashing

Materia: Organización y Estructura de Datos | Autores: Carlos Rico · Valeria A. Di Croce

1. ¿Qué es la Organización Directa de Archivos?

La organización directa (o de acceso directo) permite localizar cualquier registro de forma inmediata, sin necesidad de recorrer los registros anteriores. A diferencia del acceso secuencial, la secuencia lógica de los registros no necesita coincidir con su ubicación física en el disco.

La ventaja principal es alcanzar, en condiciones ideales, un acceso de complejidad **O(1)**: un único acceso a disco para encontrar cualquier registro, independientemente del tamaño del archivo.

Medios de almacenamiento adecuados

- Memoria principal (RAM) y discos magnéticos: ideales para acceso directo.
- Cintas magnéticas: no son recomendables, ya que su naturaleza es estrictamente secuencial.

Estructura interna de un archivo directo

- Archivo de longitud fija y no extensible.
- Registros de longitud fija con clave primaria única.
- Cada posición tiene un atributo de estado: libre, ocupado o borrado.
- Se utiliza una función de hashing para calcular la posición de cada registro.

Pueden existir huecos libres entre registros válidos (posiciones con estado 'nulo'), lo cual es normal y esperado en esta organización.

2. Fundamentos del Acceso Directo

Para acceder directamente a un registro se necesita transformar su clave (identificador lógico) en una dirección física. Este proceso tiene tres pasos:

- **Identificación de la clave primaria:** campo o combinación de campos que identifica de manera única cada registro (DNI, número de legajo, código de producto, etc.).

- **Transformación:** aplicación de una función que convierte la clave en una dirección física de almacenamiento.
- **Acceso directo:** localización inmediata del registro en esa dirección, sin recorridos.

Requisitos para que el acceso directo sea eficiente

- Las claves deben mantener un orden ascendente con pocos valores sin usar, para minimizar el desperdicio de espacio.
- Lo ideal es que cada clave se corresponda directamente con una dirección física.
- Debe existir exactamente una dirección por cada valor posible de la clave, sin duplicados.
- Los valores de las claves deben estar dentro de un rango limitado y predecible.

Comparativa de rendimiento

Para un archivo de 10.000 registros, la diferencia entre métodos de acceso es enorme: la búsqueda secuencial ordenada requiere un promedio de 13 accesos ($\log_2$ de 10.000), los árboles balanceados necesitan menos de 3, y el hashing ideal logra acceso en 1 solo paso.

3. Introducción al Hashing

El hashing (o dispersión) es la técnica fundamental de la organización directa. Consiste en aplicar una función matemática a la clave del registro para obtener directamente su dirección de almacenamiento, sin necesidad de índices separados.

Formalmente: $H: X \rightarrow D$, donde X es el conjunto de claves posibles, D es el conjunto de direcciones, y $H(x)$ es la función de transformación. Es una aplicación suprayectiva: todo espacio de direcciones puede ser alcanzado.

Origen histórico

El término 'hash' proviene de su significado en inglés: picar y mezclar. Donald Knuth atribuye su primera utilización a H.P. Luhn de IBM en enero de 1953, aunque su uso masivo no comenzó hasta la década de 1960.

Tipos de claves

- **Claves numéricas:** DNI, números de legajo, identificadores numéricos. Se usan directamente.
- **Claves textuales:** apellidos, nombres de productos. Requieren conversión a números antes de aplicar la función hash.

Operaciones básicas del TAD TablaHash

- **Inicializar:** crea una tabla hash vacía.
- **Hash(clave):** devuelve la posición calculada a partir de la clave.

- **Buscar(tabla, clave):** retorna el atributo asociado si la clave existe; error en caso contrario.
- **Insertar(tabla, clave, atributo):** almacena el par en la posición calculada; error si hay colisión sin resolver.
- **Eliminar(tabla, clave):** marca la posición como libre; error si la clave no existe.

4. Métricas de Rendimiento: Densidad y Factor de Carga

Dos métricas son fundamentales para evaluar y diseñar una tabla hash:

Densidad de claves

Mide qué tan 'lleno' está el universo de claves posibles respecto al espacio asignado. Se calcula como la cantidad de registros reales (R) dividida entre el universo de claves posibles (|X|). Si la densidad es muy baja, el acceso directo desperdicia demasiada memoria y conviene usar hashing.

Factor de carga (alpha)

Mide cuán cargada está la tabla hash. Se calcula como R / N , donde R es el número de registros insertados y N es el número de direcciones (buckets) disponibles. Si cada bucket puede guardar C registros, el factor efectivo es $R / (N \times C)$.

Regla práctica: a mayor factor de carga (cercano a 1 o superior), mayor probabilidad de colisiones. Un factor de carga de 0.75 es un valor comúnmente recomendado para equilibrar espacio y rendimiento.

Probabilidad de colisiones: distribución de Poisson

Como el hashing distribuye registros de forma pseudoaleatoria, la probabilidad de que exactamente X registros caigan en la misma dirección se modela con la distribución de Poisson: $p(x) = ((R/N)^x * e^{-(R/N)}) / x!$. Por ejemplo, con $N=1000$ y $R=1000$ (densidad=1): $P(2 \text{ colisiones})=0.184$, $P(3)=0.061$, $P(4)=0.015$.

5. Técnicas de Construcción de Funciones Hash

Una buena función hash debe distribuir las claves de manera uniforme en el espacio de direcciones para minimizar colisiones. Existen cinco métodos principales:

5.1 Resto de División (Módulo)

Es el método más simple y más usado: $h(k) = k \bmod m$, donde k es la clave y m es el tamaño de la tabla.

- Ejemplo: $k=123456$, $m=1009 \rightarrow h(k) = 123456 \bmod 1009 = 358$.

- **Recomendación clave:** elegir m como un número primo que no sea potencia de 2 ni múltiplo de 10. Esto evita patrones de colisión regulares y distribuye mejor los residuos.

Atención: Evitar $m =$ potencias de 2, ya que $h(k)$ dependería solo de los bits de menor orden de k , produciendo distribuciones sesgadas.

5.2 Multiplicación (método de Knuth)

Fórmula: $h(k) = \text{floor}(m * (\{k * A\}))$, donde $\{x\}$ es la parte fraccionaria de x , y A es una constante irracional recomendada: $A = (\text{raiz}(5) - 1) / 2 \approx 0.6180339887$ (número áureo). m suele ser una potencia de 2.

- Ejemplo con DNI $k=35678789$, $m=10000$: $k*A=22053235.877 \rightarrow$ parte fraccionaria $=0.877 \rightarrow h=\text{floor}(10000*0.877)=8770$.
- **Ventaja:** menos sensible a patrones regulares en las claves; funciona bien con m potencia de 2.

5.3 Plegado (Fold & Add)

Se divide la clave en bloques del mismo tamaño y se suman. Al final se aplica módulo m . Para claves textuales, se convierte primero a números (ASCII/UTF-8).

- **Variante A — Shift Folding:** se suman los bloques tal como están. Ej: $k=12345678 \rightarrow [12,34,56,78] \rightarrow 180 \bmod 100 = 80$.
- **Variante B — Boundary Folding:** se alternan bloques normales e invertidos antes de sumar. Ej: $[12,43,56,87] \rightarrow 198 \bmod 100 = 98$. Produce mayor mezcla.
- Ejemplo con email 'pepe@gmail.com', tabla de 10.000 posiciones: valores ASCII en bloques de 4 $\rightarrow$ suma $=34380 \rightarrow 34380 \bmod 10007 = 4359$.

5.4 Compresión (XOR)

Se divide la clave en componentes, se traducen a binario y se aplica la operación XOR acumulada entre todos los bytes. Al resultado se aplica módulo m .

- Ejemplo: $C='David' \rightarrow D=01100100, A=01100001, V=01110010, I=01101001, D=01100100 \rightarrow \text{XOR acumulado}=01111010=122 \rightarrow 122 \bmod 26 = 18$.
- **Uso típico:** claves alfanuméricas donde se busca simplicidad computacional.

5.5 Cuadrado Medio (Mid-Square)

Se eleva la clave al cuadrado y se extraen los dígitos del centro del resultado. La cantidad de dígitos extraídos determina el rango de la tabla.

- Ejemplo: $k=35678789$, tabla 4 dígitos $\rightarrow k^2=1272913441351121$ (16 dígitos) $\rightarrow$ posición 7 a 10 $= 3441$.
- Los dígitos centrales mezclan la influencia de todos los dígitos de la clave, produciendo buena distribución.

6. Colisiones y Desbordamiento

Una **colisión** ocurre cuando dos claves distintas producen la misma dirección hash: $f(k_1) = f(k_2)$ con $k_1 \neq k_2$. Estas claves se denominan **sinónimas**. Un **desbordamiento** (overflow) ocurre cuando se intenta insertar en una dirección completamente ocupada. Cuando cada dirección tiene una sola celda, ambos suceden simultáneamente.

El algoritmo de hash perfecto no existe en la práctica general. Hay que diseñar con colisiones en mente y elegir estrategias de resolución adecuadas.

Estrategias generales para reducir colisiones

- **Propagar los registros:** usar funciones hash que distribuyan aleatoriamente, evitando agrupaciones.
- **Usar memoria extra:** reservar más espacio del estrictamente necesario (factor de carga bajo).
- **Cubetas (buckets):** permitir que cada dirección almacene C registros en lugar de uno solo.

Hashing perfecto vs. imperfecto

- **Perfecto:** no produce colisiones. Solo posible cuando el espacio de claves se conoce completamente de antemano.
- **Imperfecto:** puede producir colisiones. Es el caso general en la práctica.

7. Métodos de Resolución de Colisiones

Existen dos grandes familias de técnicas para resolver colisiones: el direccionamiento cerrado (hashing abierto o encadenamiento) y el direccionamiento abierto (hashing cerrado o sondeo).

7.1 Direccionamiento Abierto (Progressive Overflow / Sondeo)

Todos los registros se almacenan dentro de la tabla principal. Cuando hay colisión, se busca otra celda libre mediante alguna estrategia de sondeo.

- **Sondeo Lineal:** $P_i = (f(k) + i \cdot c) \bmod m$, con c constante (típicamente $c=1$). Algoritmo: calcular $h(k)$; si está libre, insertar; si no, avanzar a $(i+1) \bmod m$; repetir hasta encontrar celda libre o volver al inicio.
- **Sondeo Cuadrático:** $P_i = (f(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$. Reduce el 'clustering' del sondeo lineal pero puede dejar posiciones sin explorar.
- **Doble Hashing:** $P_i = (f(k) + i \cdot g(k)) \bmod m$, donde $g(k)$ es una segunda función hash que determina el salto. Es la técnica más robusta del grupo: elimina el clustering primario y secundario.

Ejemplo de doble hashing: $m=7$, $h_1(k)=k \bmod 7$, $h_2(k)=1+(k \bmod 6)$. Para $k=32$: $h_1=4$ (ocupado) $\rightarrow$ siguiente: $(4 + 1 \cdot h_2(32)) \bmod 7 = (4+3) \bmod 7 = 0$. Se inserta en posición 0.

Atención: En sondeo lineal, la eliminación requiere marcar las celdas como 'borradas' (no simplemente vacías), ya que una celda vacía interrumpiría la cadena de búsqueda de sinónimos posteriores.

7.2 Encadenamiento con Área de Overflow Separada

Los registros que colisionan se almacenan en un área de overflow separada del área principal. Cada posición del área principal guarda solo un puntero (la cabeza de una lista enlazada). La búsqueda va al home address en el área principal y sigue la cadena en el área de overflow.

7.3 Encadenamiento con Progressive Overflow (Coalesced Hashing)

Los registros sinónimos se encadenan con punteros dentro del mismo archivo. Cada registro guarda: dirección hogar, dirección actual y puntero al siguiente sinónimo. Si la celda hogar está ocupada, el nuevo registro se coloca en la siguiente posición libre y se actualiza el puntero del sinónimo anterior.

7.4 Almacenamiento por Cubetas (Bucket Hashing)

En lugar de asignar una celda por dirección, cada dirección es un bloque (cubeta) que puede almacenar C registros. El factor de carga efectivo es $\alpha_{ef} = R / (N \cdot C)$. Varias claves que hashan al mismo índice conviven en la cubeta sin colisión hasta que se llena.

- Si una cubeta se desborda, se asigna una cubeta de overflow y se encadena con un puntero desde la cubeta principal.
- Si se usa direccionamiento abierto a nivel de cubeta, cuando una está llena se prueba la siguiente cubeta disponible.

7.5 Scatter Tables (Tablas Dispersas)

No existe un área primaria de datos como tal; solo existe una tabla de punteros a listas de registros sinónimos. $h(K)$ selecciona una ranura de la tabla, que apunta a la lista donde residen los registros con esa dirección hogar.

- **Pros:** flexible, fácil borrado, crece dinámicamente.
- **Contras:** peor localidad de caché (punteros dispersos), múltiples E/S en disco, implementación compleja.

8. Hash Extendido (Extensible Hashing)

El hash extendido resuelve una de las limitaciones más críticas del hashing clásico: permite que el archivo **crezca dinámicamente** sin necesidad de cambiar la función de hashing ni reorganizar todo el archivo.

Estructuras requeridas

- **Tabla de dispersión:** almacena su tamaño (tt) y contiene punteros a bloques de datos. Puede crecer duplicándose.

- **Archivo de bloques:** cada bloque tiene capacidad fija para registros de longitud fija y un tamaño local de dispersión (td) que indica cuántas entradas de la tabla apuntan a él.
- **Lista de bloques libres:** administra los bloques disponibles para reutilización.
- **Función de hashing:** generalmente $h(\text{clave}) = \text{clave} \bmod \text{tabla.td}$, usando los bits sufijos de la clave.

Funcionamiento general

Cuando un bloque se llena, se divide: se duplica la tabla (si es necesario) y los registros del bloque se redistribuyen entre dos nuevos bloques. Solo el bloque desbordado se divide, no todo el archivo.

Ventajas y limitaciones

- **Ventaja:** búsquedas de alta velocidad y crecimiento dinámico comparable al B-Tree.
- **Ventaja:** acceso en tiempo constante para búsquedas puntuales.
- **Limitación:** para acceso secuencial ordenado, el B+Tree sigue siendo superior.
- **Limitación:** carece de métricas directas sobre la capacidad de cada cubeta; puede tener tabla muy grande si hay muchas divisiones.

9. Otras Aplicaciones del Hashing

El hashing no se limita a la organización de archivos. Como función unidireccional (no se puede revertir), tiene aplicaciones críticas en seguridad:

- **Integridad de documentos:** cualquier cambio en el documento produce un hash completamente diferente.
- **Firmas digitales:** se hashea el documento y se firma el resumen, no el documento completo.
- **Verificación de contraseñas:** los sistemas almacenan el hash, nunca la contraseña en claro.
- **Sellos de tiempo:** acreditar que un documento existía en un momento determinado.

Algoritmos criptográficos de hash

- **MD5:** produce resúmenes de 128 bits. Hoy considerado inseguro para aplicaciones criptográficas, aunque aún se usa para verificación de integridad no crítica.
- **SHA (Secure Hash Algorithm):** familia de algoritmos de 160 bits o más (SHA-1, SHA-256, SHA-512). SHA-256 es el estándar actual más utilizado.

10. Resumen y Conclusiones

Acceso directo: permite localizar cualquier registro sin recorridos, con complejidad $O(1)$ ideal. Requiere función de transformación clave $\rightarrow$ dirección.

Funciones hash: las cinco técnicas vistas (módulo, multiplicación, plegado, compresión, cuadrado medio) tienen distintas ventajas. En la práctica, el módulo con primo y la multiplicación con la constante áurea son las más utilizadas.

Factor de carga: es la métrica clave de diseño. Un factor bajo reduce colisiones pero desperdicia espacio; uno alto ahorra espacio pero degrada el rendimiento. El punto de equilibrio habitual es alrededor de 0.75.

Resolución de colisiones: el sondeo lineal es simple pero sufre clustering; el doble hashing elimina este problema; el encadenamiento por cubetas reduce colisiones con buckets de capacidad C ; las scatter tables son las más flexibles pero las más complejas.

Hash extendido: solución moderna para archivos que crecen dinámicamente, comparable al B-Tree en búsquedas puntuales pero inferior para acceso secuencial.

Conclusión general: el hashing es la técnica de acceso más rápida para búsquedas puntuales. Su mayor debilidad es la gestión de colisiones y la dificultad para el acceso secuencial ordenado. Para aplicaciones que requieran ambas cosas (búsqueda rápida Y recorrido ordenado), los árboles B+ siguen siendo la elección preferida.